AUTOMOTIVE LARAVEL AGENTS

Course Name: **Datagami Skill Based Course – Agentic AI**

Institution Name: Medicaps University

Sr no	Student Name	Enrolment Number
1.	Yashit Jain	EN22EL301069
2.	Yugal Bisani	EN22EL301070
3.	Yashasvi Nannaware	EN22EL301068
4.	Yuvraj Singh Sikarwar	EN22CS3011129

Group Name: 03-D10

Project Number: GAI-16:

Industry Mentor Name: Mr Ashwin

University Mentor Name: Mrs Maya Yadav Academic

Year:2026

TABLE OF CONTENTS

1. **INTRODUCTION**
   - 1.1 Overview
   - 1.2 Motivation
   - 1.3 Objectives
2. **PROBLEM STATEMENT & OBJECTIVES**
   - 2.1 Problem Statement
   - 2.2 Project Objectives
   - 2.3 Scope of the Project
   - 2.4 Limitations
3. **LITERATURE REVIEW**
   - 3.1 Existing Systems
   - 3.2 Need for Proposed System
4. **PROPOSED SYSTEM**
   - 4.1 System Overview
   - 4.2 Key Features
   - 4.3 Workflow
5. **SYSTEM DESIGN**
   - 5.1 Architecture Design
   - 5.2 System Architecture Diagram
   - 5.3 Process Flow
   - 5.4 Information Flow
   - 5.5 Component Design
   - 5.6 Data Flow
6. **TOOLS & TECHNOLOGIES**
   - 6.1 Software Tools
   - 6.2 Hardware Requirements
7. **RESULTS & OUTPUT**
   - 7.1 Screenshots
8. **CONCLUSION**
   - 8.1 Summary
   - 8.2 Learning Outcomes
9. **FUTURE SCOPE**
   - 9.1 Enhancements
10. **REFERENCES**
    - 10.1 Sources

1. INTRODUCTION

1.1 Overview

Automotive Laravel Agents is an advanced multi-agent AI web application that leverages the power of Large Language Models (LLMs) to generate comprehensive, structured automotive reports from simple natural language queries. The system is designed to bridge the gap between user curiosity and expert-level automotive knowledge by automating the research and report-writing process entirely.

The application is built using React and integrates two specialized AI agents powered through the Groq API — a Research Agent and a Writer Agent — working in a sequential pipeline. It provides a fast, interactive, and visually engaging platform where users can generate detailed reports on any car, EV, or automotive topic in seconds.

This project demonstrates how AI agents can automate complex multi-step tasks and deliver structured, high-quality outputs that would otherwise require hours of manual research and writing.

1.2 Motivation

Researching automotive topics typically requires browsing multiple sources, reading technical documentation, and manually synthesizing information into a coherent report. This process is time-consuming, inconsistent in quality, and demands domain expertise.

The motivation behind this project includes:

- Reducing dependency on manual research and multiple browser tabs
- Providing a platform for instant, structured automotive report generation
- Demonstrating the practical power of multi-agent AI pipelines
- Making professional-quality automotive research accessible to all users
- Exploring prompt engineering and agent orchestration in a real-world contex

1.3 Objectives

The major objectives of this project are:

- To develop a multi-agent AI web application for automotive report generation
- To implement a two-agent pipeline using specialized LLM models via the Groq API
- To generate structured reports with clearly defined sections automatically
- To provide real-time processing with an interactive and animated dashboard UI
- To ensure ease of use with a clean, single-page React interface
- To demonstrate agent orchestration, prompt engineering, and API integration

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Generating a comprehensive automotive report is a complex, time-consuming, and knowledge-intensive task. It requires users to browse multiple sources, understand technical specifications, and manually organize information into a readable format. Many users, especially students and enthusiasts, do not have access to domain expertise or tools capable of doing this automatically.

There is a need for a system that simplifies this process by allowing users to generate detailed automotive reports instantly using artificial intelligence. The goal is to create a solution that reduces time, effort, and expertise requirements while maintaining high-quality, structured output.

2.2 Project Objectives

- To build a multi-agent AI-powered report generation system
- To provide both research and structured writing functionality through separate agents
- To ensure fast and efficient processing of user queries via the Groq LPU API
- To create a scalable and flexible single-page application architecture
- To improve user experience through an animated, interactive dashboard interface
- To integrate multiple technologies including React, Groq API, and Canvas animation

2.3 Scope of the Project

The scope of the project includes:

- Generating automotive reports using a two-agent AI pipeline
- Supporting any natural language query related to cars, EVs, or automotive technology
- Providing real-time report generation with card-based HTML output
- Running entirely client-side — no backend server or database required
- Delivering a visually engaging animated dashboard experience

2.4 Limitations:

- The system is focused on automotive topics only
- Performance depends on Groq API availability and internet connectivity
- API key is currently exposed client-side without a backend proxy
- No report export functionality (PDF/Word download) is currently implemented

3. LITERATURE REVIEW

3.1 Existing Systems

There are several AI-powered tools available such as ChatGPT, Perplexity AI, and Google Gemini that can answer automotive questions and generate text content. These tools, however, are general-purpose systems and not specifically designed for structured automotive report generation. They operate on a single-prompt basis and do not chain multiple specialized agents to improve output quality and structure

3.2 Need for Proposed System

The existing tools lack structured multi-agent orchestration for automotive report generation. Therefore, there is a need for a system that:

- Focuses specifically on automotive research and structured report writing
- Uses separate specialized agents for research and writing phases
- Delivers output in a formatted, section-based report layout
- Is easy to use with a clean and visually engaging interface
- Offers faster and higher-quality outputs through model specialization

4. PROPOSED SYSTEM

4.1 System Overview

The proposed system is a multi-agent AI web application that generates structured automotive reports from user queries. It follows a Sequential Multi-Agent Pipeline Architecture where a React-based Single Page Application (SPA) serves as the frontend, and all AI intelligence is delegated to two specialized LLM agents accessed via the Groq REST API.

The system ensures smooth orchestration between the Research Agent and Writer Agent, and provides real-time rendered results with an animated, interactive dashboard.

4.2 Key Features

- Two-agent sequential pipeline (Research Agent + Writer Agent)
- Natural language query input for any automotive topic
- Structured 4-section report generation (Executive Summary, Features, Specs, Conclusion)
- Real-time report rendering in a card-based HTML layout
- Animated canvas background with Brownian motion car animation
- Dashboard-style UI with agent sidebar showing active agents
- Entirely client-side — no backend server required

4.3 Workflow

The working process of the system is as follows:

- Step 1: The user enters an automotive query (e.g., 'Tesla Model S') on the dashboard
- Step 2: The run() function sets loading state and clears previous output
- Step 3: Agent 1 (Research Agent — llama-3.3-70b) is called with a broad analysis prompt
- Step 4: Agent 1 returns comprehensive research content about the queried topic
- Step 5: Agent 2 (Writer Agent — llama-3.1-8b-instant) is called with Agent 1's output and a structured template
- Step 6: Agent 2 produces a formatted 4-section report
- Step 7: The format() function parses the response into HTML report cards
- Step 8: The UI re-renders and displays the completed report dynamically

5. SYSTEM DESIGN

5.1 Architecture Design

The system architecture follows a Multi-Agent Pipeline Architecture with a React-based Single Page Application as the presentation layer. It consists of three logical layers: the Presentation Layer (React UI + Canvas animation), the Application Layer (React state management and API orchestration), and the Agent Orchestration Layer (sequential LLM agent calls via Groq REST API). All components work together to process user queries and deliver structured report output efficiently.

5.2 System Architecture Diagram

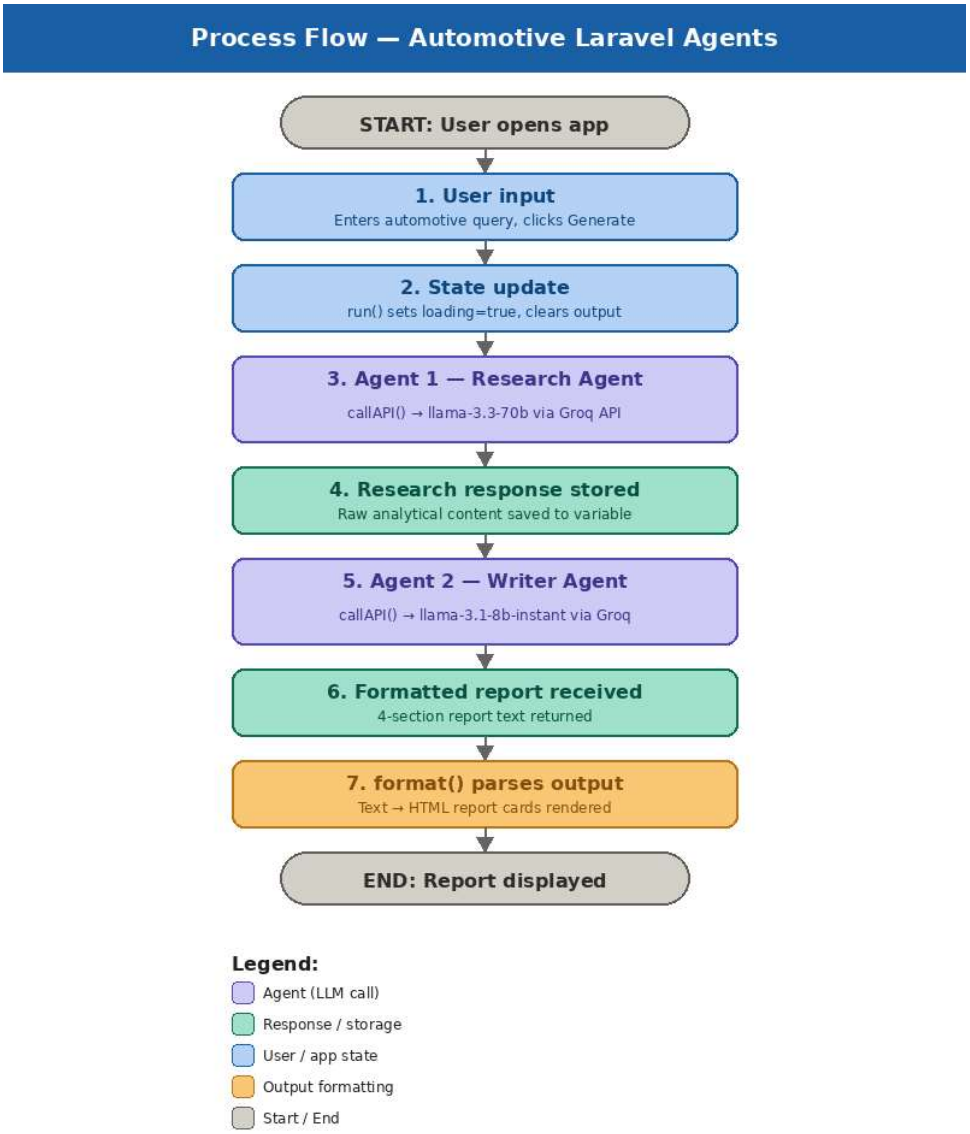
The system architecture is structured across three layers — Presentation, Application, and Agent Orchestration — with data flowing from the user query through the React SPA to the Groq API agents and back as a rendered HTML report.

LAYER	COMPONENTS
Presentation Layer	index.html, style.css, background.js (Canvas Animation)
Application Layer	app.js (React SPA) — Landing Page, Dashboard, State Management
Agent Orchestration	callAPI() → Agent 1 (Research) → Agent 2 (Writer) → format()
External API	Groq REST API — llama-3.3-70b-versatile & llama-3.1-8b-instant

5.3Process Flow

The diagram below describes the step-by-step process from the moment a user opens the app to receiving a generated report. Decision points (loading state, agent switching) and the sequential pipeline are explicitly shown:

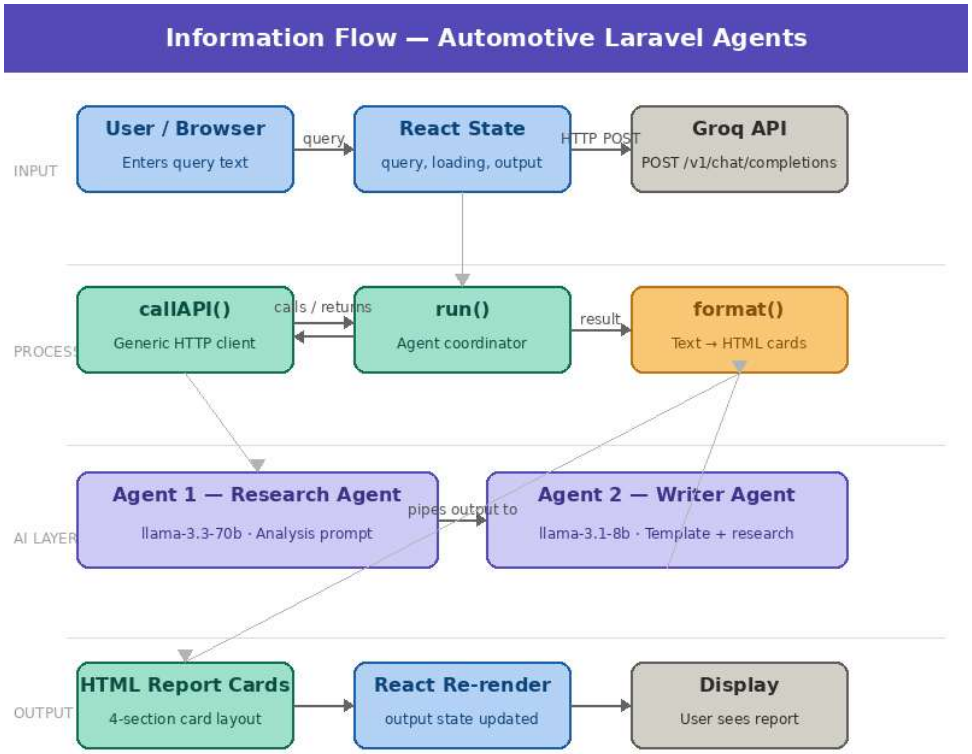
- START: User opens the React SPA landing page
- User Input: User enters an automotive topic query and clicks 'Generate Report'
- State Update: run() sets loading=true and clears previous output
- Agent 1 Call: callAPI() sends research prompt to llama-3.3-70b via Groq API
- Agent 1 Response: Raw analytical research content returned and stored
- Agent 2 Call: callAPI() sends structured template prompt + Agent 1 output to llama-3.1-8b
- Agent 2 Response: Formatted 4-section report text returned
- Formatting: format() parses text into HTML report cards
- END: UI re-renders and displays the report; loading=false



5.4 Information Flow

The diagram below maps how data moves between every node — user, React state, Groq API agents, and the rendered output:

- Data In: User query string (text input)
- Processing: React state management → callAPI() → Groq REST API → LLM inference
- AI Output: Raw research text (Agent 1) → Formatted report text (Agent 2)
- Render: format() converts text → HTML → dangerouslySetInnerHTML renders report cards



5.2 Component Design

The system is divided into the following components:

- User Interface (React SPA): Handles user interaction, state management, and report rendering
- Landing Page Component: Hero section with project title, tagline, and entry button
- Dashboard Component: Two-panel layout with Agent Sidebar and Main Content area
- Agent Orchestration (run() + callAPI()): Manages sequential agent calls and result piping
- Output Formatter (format()): Converts LLM text output into structured HTML report cards
- Canvas Animation (background.js): Runs an independent real-time car animation loop

5.3 Data Flow

The data flow in the system follows a structured sequential path:

- User input (query string) is captured by the React controlled input component
- The run() function orchestrates the two-agent pipeline via callAPI()
- Agent 1 processes the query and returns analytical research content
- Agent 2 receives Agent 1's output and returns a structured 4-section report
- The format() function parses and renders the output as HTML report cards

6. TOOLS & TECHNOLOGIES

6.1 Software Tools

UI Framework	React 18 (CDN)	Component-based UI with hooks (useState)
Transpiler	Babel Standalone (CDN)	In-browser JSX transpilation without build step
Styling	Vanilla CSS	Custom dark-theme dashboard with glassmorphism effects
Animation	HTML5 Canvas API	Real-time car animation with Brownian motion
API Client	Fetch API (native)	HTTP calls to Groq REST endpoint
LLM Provider	Groq API (LPU)	Ultra-fast inference for both AI agents
Agent 1 Model	llama-3.3-70b-versatile	Deep research and analytical reasoning
Agent 2 Model	llama-3.1-8b-instant	Fast structured report writing
Fonts	Google Fonts — Inter	Modern sans-serif typography
Hosting	Static HTML	No server required — runs from any static host

6.2 Hardware Requirements

- Minimum 4GB RAM (8GB recommended for smooth browser performance)
- Stable internet connection (required for Groq API calls)
- Modern web browser (Chrome, Firefox, Edge, Safari)
- No GPU required — all AI inference is handled server-side by Groq LPUs

7. RESULTS & OUTPUT

The application delivers the following key screens during a typical user session:

- Landing Page: Hero section displaying the project title 'Automotive Laravel Agents', tagline, and a feature strip highlighting Smart Analysis, Multi-Agent, and AI Reports capabilities.

- Dashboard View: Two-panel layout showing the Agent Sidebar on the left (displaying Research Agent and Writer Agent as active visual cards) and the Main Panel on the right (containing the query input field and report output area).

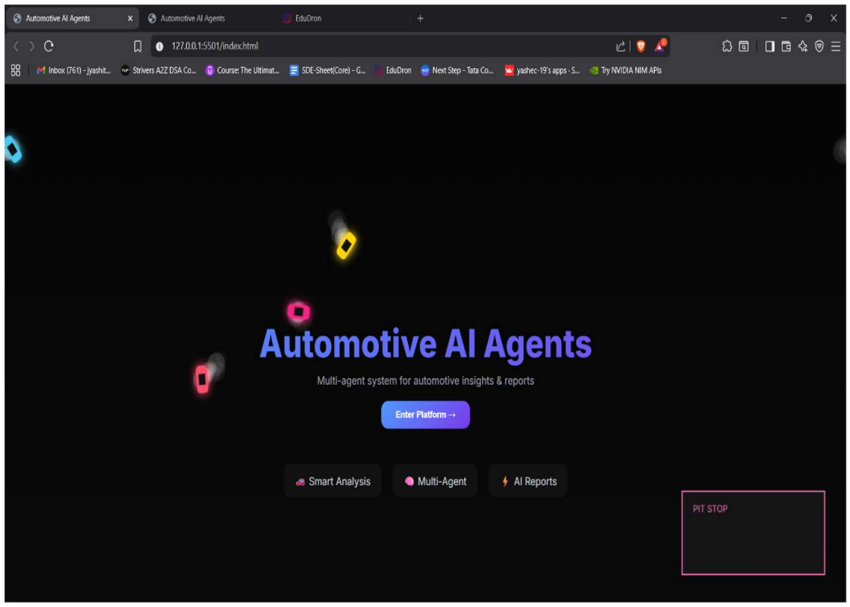
- Animated Background: Full-screen canvas animation with four colored cars (red, cyan, pink, yellow) moving with Brownian angular drift, emitting smoke particles, and a Pit Stop overlay in the corner.

- Generated Report Output: A structured multi-card report layout with sections for Executive Summary, Key Features, Technical Specifications, and Conclusion — rendered dynamically for any automotive topic query.

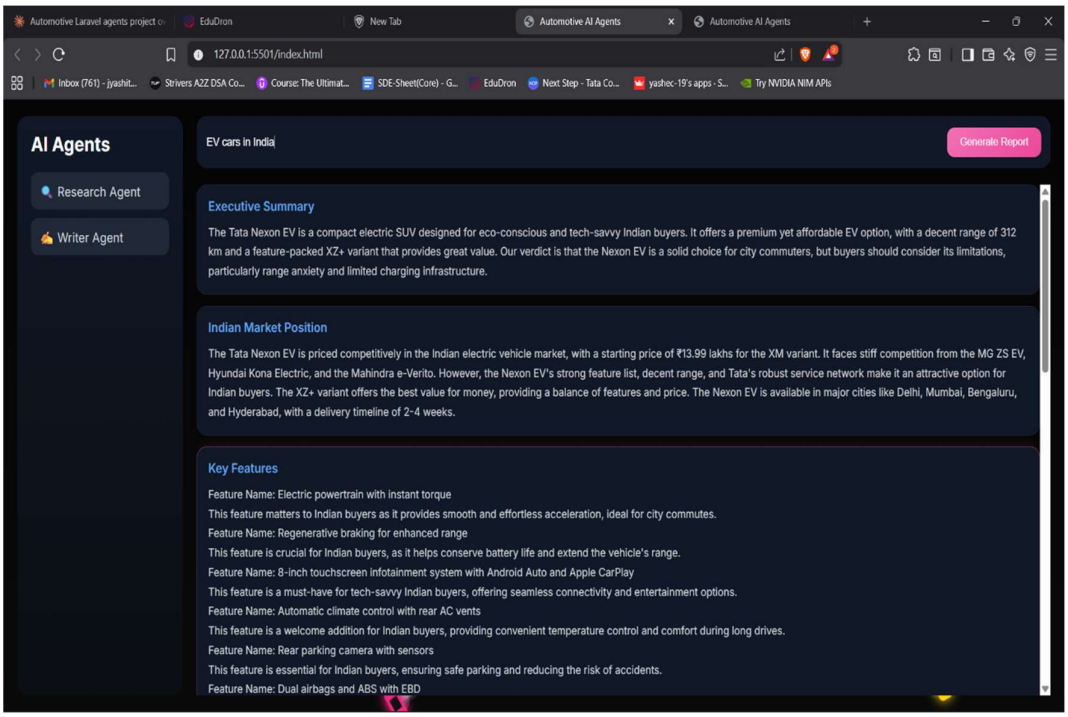
  Example query: 'Tesla Model S' → The system generates a full structured report covering the vehicle's overview, key features such as Autopilot and Ludicrous Mode, technical specs including range and performance figures, and a conclusion, all within 5–15 seconds.

7.1 Screenshots

- **Home page**



- **Output**


Automotive Laravel agents project
EduDron
New Tab
Automotive AI Agents
Automotive AI Agents
127.0.0.1:5501/index.html
Inbox (761) - jyashit...
Strivers A2Z DSA Co...
Course: The Ultimat...
SDE-Sheet(Core) - G...
EduDron
Next Step - Tata Co...
yashec-19's apps - S...
Try NVIDIA NIM APIs
AI Agents
Research Agent
Writer Agent
EV cars in India
Generate Report
Executive Summary
The Tata Nexon EV is a compact electric SUV designed for eco-conscious and tech-savvy Indian buyers. It offers a premium yet affordable EV option, with a decent range of 312 km and a feature-packed XZ+ variant that provides great value. Our verdict is that the Nexon EV is a solid choice for city commuters, but buyers should consider its limitations, particularly range anxiety and limited charging infrastructure.
Indian Market Position
The Tata Nexon EV is priced competitively in the Indian electric vehicle market, with a starting price of ₹13.99 lakhs for the XM variant. It faces stiff competition from the MG ZS EV, Hyundai Kona Electric, and the Mahindra e-Verito. However, the Nexon EV's strong feature list, decent range, and Tata's robust service network make it an attractive option for Indian buyers. The XZ+ variant offers the best value for money, providing a balance of features and price. The Nexon EV is available in major cities like Delhi, Mumbai, Bengaluru, and Hyderabad, with a delivery timeline of 2-4 weeks.
Key Features
Feature Name: Electric powertrain with instant torque
This feature matters to Indian buyers as it provides smooth and effortless acceleration, ideal for city commutes.
Feature Name: Regenerative braking for enhanced range
This feature is crucial for Indian buyers, as it helps conserve battery life and extend the vehicle's range.
Feature Name: 8-inch touchscreen infotainment system with Android Auto and Apple CarPlay
This feature is a must-have for tech-savvy Indian buyers, offering seamless connectivity and entertainment options.
Feature Name: Automatic climate control with rear AC vents
This feature is a welcome addition for Indian buyers, providing convenient temperature control and comfort during long drives.
Feature Name: Rear parking camera with sensors
This feature is essential for Indian buyers, ensuring safe parking and reducing the risk of accidents.
Feature Name: Dual airbags and ABS with EBD

8. CONCLUSION

8.1 Summary

The Automotive Laravel Agents project successfully demonstrates the core principles of AI Agents and multi-agent system design in a practical, domain-specific application. By decomposing a complex task — generating a comprehensive automotive report — into two specialized agents with different models and roles, the system achieves better output quality and clarity than a single-prompt approach would provide.

The project showcases key AI engineering concepts including prompt engineering, agent orchestration, API integration, asynchronous state management in React, and real-time canvas animation — all within a lightweight, dependency-minimal architecture that runs entirely in the browser.

8.2 Learning Outcomes

- Understanding of AI Agents and multi-agent system design principles
- Hands-on experience with prompt engineering and LLM orchestration
- Practical knowledge of React 18, hooks, and SPA architecture
- Integration of external REST APIs (Groq) with frontend applications
- Experience with HTML5 Canvas API for real-time animation
- Understanding of model selection trade-offs (large vs. fast LLMs)

9. FUTURE SCOPE

9.1 Enhancements

- Add a third Comparison Agent that can compare two vehicles side by side
- Implement a backend proxy (Node.js / Python FastAPI) to secure the Groq API key
- Add PDF and Word export functionality so users can save generated reports
- Introduce streaming responses so report text appears progressively as the LLM generates it
- Add a memory layer so the system can recall previously generated reports
- Integrate real-time web search tools so agents can retrieve live automotive news and pricing
- Make the UI fully mobile-responsive for use on smartphones and tablets
- Develop a mobile application version for iOS and Android

10. REFERENCES

10.1 Sources

- Groq API Documentation — https://console.groq.com/docs
- Meta LLaMA 3 Model Card — https://ai.meta.com/llama
- React 18 Official Documentation — https://react.dev
- HTML5 Canvas API — https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- OpenAI Chat Completions API Schema — https://platform.openai.com/docs/api-reference/chat
- Wooldridge, M. (2009). An Introduction to MultiAgent Systems. John Wiley & Sons.
- Hugging Face Documentation — https://huggingface.co/docs
- Online tutorials and AI Agents research articles — various sources